Ad Soyad: _____	**Tarih:** _____
Lab 12 – Hafta 12 – *Ağ Programlamaya Giriş: POSIX Echo, Qt TCP İstemci, Length-Prefix Chat ve JSON Dispatch*	

Lab Hakkında

Bu laboratuvar çalışmasında Bölüm 12'de ele alınan ağ programlama temellerini uygulamalı deneyeceksiniz. Önce POSIX seviyesinde bir TCP echo sunucusu/istemcisi yazacak, sonra Qt'nin asenkron `QTcpSocket` sınıfını kullanarak istemci yazacak, ardından `QTcpServer` + length-prefix çerçeveleme ile çoklu istemci destekli mini chat sunucusu kuracaksınız. Bonus çalışmada gelen mesajları JSON olarak parse edip komut bazlı cevaplar üreteceksiniz.

Süre: 3 saat **Gereksinimler:** g++ (C++20 desteği), Qt 6 (Core, Network, Widgets), `nc` veya `telnet`

Derleme Komutları

Saf POSIX (Çalışma 1) için:

```
g++ -std=c++20 -O2 -o echo_sunucu echo_sunucu.cpp
g++ -std=c++20 -O2 -o echo_istemci echo_istemci.cpp
```

Qt projeleri (Çalışma 2, 3 ve Bonus) için CMake:

```
cd calismaN_xxx
cmake -B build -DCMAKE_PREFIX_PATH=/path/to/Qt6
cmake --build build
```

Lab başlangıç kodları: `out/LabExamples/Lab12_Ag_Programlamaya_Giris/` dizininde mevcut. Her dosyada `// GOREV-N` yorumlarını dolduracaksınız.

1 Çalışma 1: POSIX TCP Echo Sunucu/İstemci (45 dk)

Ön Bilgi

Bölüm 12 Ünite 2'de POSIX soket API'sini öğrendik: `socket()` bir dosya tanıtıcısı döner; `bind` sokete adres verir, `listen` dinlemeye geçirir, `accept` istemci bağlantısını kabul eder. `recv` ve `send` ile veri akar. Bu çalışmada bu çağrıları doğrudan kullanarak ham bir echo sunucusu yazacaksın.

Adım 1: İskelet Kodu İncele

`calisma1_posix/echo_sunucu.cpp` ve `calisma1_posix/echo_istemci.cpp` dosyalarını aç. Her iki dosyada `// GOREV 1` ve `// GOREV 2` yorum işaretlerini bulacaksın.

Listing 1: Sunucu Yaşam Döngüsü Hatırlatma

```
int dinleme = ::socket(AF_INET, SOCK_STREAM, 0);
int evet = 1;
::setsockopt(dinleme, SOL_SOCKET, SO_REUSEADDR, &evet, sizeof(evet));
sockaddr_in adres{};
adres.sin_family      = AF_INET;
adres.sin_port        = htons(9000);
adres.sin_addr.s_addr = htonl(INADDR_ANY);
::bind(dinleme, reinterpret_cast<sockaddr*>(&adres), sizeof(adres));
::listen(dinleme, 16);
// while (true) { int istemci = ::accept(...); ... }
```

Adım 2: Görevler (Sunucu)

1. **GOREV 1**: `echo_sunucu.cpp`'de soketi yarat, `SO_REUSEADDR` koy, 9000 portuna `bind` et, `listen` çağır. Her hatada `strerror(errno)` ile mesaj yazdırıp dön.
2. **GOREV 2**: `accept` döngüsü kur. Her kabul edilen istemcide `recv`/`send` echo döngüsünü yaz. `recv` 0 dönerse istemci kapattı, çık. `recv` 0'dan büyük olan baytları geri yaz; kısmi yazmaya karşı iç içe `send` döngüsü kullan.

Adım 3: Görevler (İstemci)

1. **GOREV 1**: `echo_istemci.cpp`'de soket yarat, `inet_pton` ile `127.0.0.1` adresini hazırla, `connect` çağır.
2. **GOREV 2**: stdin'den satır oku, sokete yolla, `recv` ile cevabı al, ekrana yazdır.

Adım 4: Test

```
# Bir terminalde:
./echo_sunucu

# Baska terminalde (kendi istemcin):
./echo_istemci

# Veya Unix klasik aracrlariyla:
nc localhost 9000
telnet localhost 9000
```

Dikkat:

`SO_REUSEADDR`'i unutursan sunucuyu Ctrl+C ile kapatip hemen tekrar baslattigında "Address already in use" hatasini alirsin (TIME_WAIT). Geliştirme döngüsünde bu seçenek neredeyse zorunludur.

Beklenen Çıktı

```
# Sunucu tarafi:
Echo sunucu 9000 portunda dinliyor...
Istemci bagli: 127.0.0.1:54321
Istemci kapatti.

# Istemci tarafi:
Sunucuya baglandi. Mesaj yazin (Ctrl+D ile cik):
merhaba
Sunucu> merhaba
```

2 Çalışma 2: Qt TCP İstemci – QTcpSocket (60 dk)

Ön Bilgi

Bölüm 12 Ünite 3 Konu 1'de gördüğümüz `QTcpSocket`, POSIX soketin sinyal/slot ile zenginleştirilmiş halidir. `connectToHost` asenkrondur; gerçek bağlantı kurulduğunda `connected` sinyali emit edilir. Veri geldiğinde `readyRead` sinyali tetiklenir; `readAll` ile baytları toplarsın.

Adım 1: Hazır Proje İskeleti

`calisma2_qt_istemci/` dizininde CMake projesi hazır:

```
calisma2_qt_istemci/
  CMakeLists.txt
  include/Istemci.h
  src/Istemci.cpp      <- GOREV 1, 2, 3 burada
  src/main.cpp
```

Adım 2: Görevler

1. **GOREV 1**: `Istemci.cpp` kurucusunda `socket_ = new QTcpSocket(this)` oluştur. `connected`, `disconnected`, `readyRead` ve `errorOccurred` sinyallerini sınıfın slot/sinyallerine bağla. `baglan` slot'unda `socket_->connectToH` `port)` çağır.
2. **GOREV 2**: `okumaHazirSlot`'unda `socket_->readAll()` ile baytları al ve `mesajGeldi` sinyali ile yukarı yolla.
3. **GOREV 3**: `mesajGonder`'de bağlantı durumunu kontrol et, `socket_->write(mesaj)` ile gönder.

Dikkat: Eski SIGNAL/SLOT Yasak

Bağlantılarda **daima yeni connect syntax'ini** kullan: `connect(socket_, &QTcpSocket::readyRead, this, &Istemci::okumaHazirSlot)`. Eski `SIGNAL(...)/SLOT(...)` makro tabanlı versiyon derleme zamanında hata yakalayamaz, yavaştır ve yıllardır deprecated kabul edilir.

Adım 3: Test

Çalışma 1'in echo sunucusunu çalıştır, ardından:

```
./qt_istemci 127.0.0.1 9000
[OK] Sunucuya baglandi: "127.0.0.1" : 9000
Mesaj yazin (Ctrl+D ile cik):
merhaba OBYS
Sunucu> merhaba OBYS
```

Dikkat: `waitFor*` Kullanma

Qt'nin `waitForReadyRead`, `waitForConnected` aile fonksiyonları çağırıldıkları thread'i bloklar. UI thread'inde dondurma yapar, iç içe olay döngüsü oluşturur. Bu lab'da *her şey* sinyal/slot tabanlı asenkron yazılmalıdır.

3 Çalışma 3: Qt TCP Server + Length-Prefix Çerçeveleme (45 dk)

Ön Bilgi

Bölüm 12 Ünite 4 Konu 1'de gördüğümüz length-prefix çerçevelemesi: her mesajın başına 4 baytlık (big-endian) uzunluk koy, sonra payload. Alıcı tarafta akümülatör buffer'a baytları topla; tam çerçeveler oluştukça çıkar. Bu çalışmada `QTcpServer` ile çoklu istemci kabul edip mesajları tüm istemcilere broadcast eden mini bir chat sunucusu yazacaksın.

Adım 1: Hazır Çerçeveleme Yardımcıları

`calisma3_qt_chat/include/cerceveleme.h` ve `src/cerceveleme.cpp` sana hazır verildi: `cerceveSar(payload)` ve `cerceveCoz(buffer)`. Önce bu dosyaları okuyup mantığını anla.

Listing 2: Çerçeveleme Yardımcısı

```
namespace cerceveleme {
    QByteArray cerceveSar(const QByteArray& payload);    // 4 bayt uzunluk + payload
    QList<QByteArray> cerceveCoz(QByteArray& buffer,     // tam cerceveleri cikar
                                 std::size_t maks_payload = 16*1024*1024);
}
```

Adım 2: Görevler

`ChatSunucu.cpp` dosyasında üç slot/yardımcı doldurulacak:

1. **GOREV 1 – yeniBaglanti**: `server_->hasPendingConnections()` döngüsünde `nextPendingConnection()` ile soket al; `istemciler_` hash'ine boş `QByteArray` buffer ile ekle. Soketin `readyRead` ve `disconnected` sinyallerini sınıfın slot'larına bağla.
2. **GOREV 2 – okumaHazir**: `sender()` ile gönderen soketi al, ilgili buffer'a `readAll` sonucunu ekle, `cerceveleme::cer` ile tam çerçeveleri çıkar; her birini `tumIstemcilereYayinla`'ya geçir. Çerçeveleme istisnası yakalarsan bağlantıyı kes.
3. **GOREV 3 – tumIstemcilereYayinla**: Tüm `istemciler_` üzerinde dolaş; her birine `cerceveSar(payload)` yaz. (Klasik chat'te gönderen kendi mesajını da görür; isteyene göre filtre koyabilirsin.)

Adım 3: Test

Length-prefix gerektiren bir istemci için saf `nc` yetmez. Aşağıdaki Python yardımcısı 4 baytlık header ekler:

```
# Iki ayri terminalde calistir, ucuncu terminalde sunucu acik:
python3 -c "
import socket, struct, sys, threading
s = socket.socket(); s.connect(('127.0.0.1', 9100))
def oku():
    while True:
        h = s.recv(4)
        if not h: break
        n = struct.unpack('>I', h)[0]
        print('<<<', s.recv(n).decode())
threading.Thread(target=oku, daemon=True).start()
for satir in sys.stdin:
    p = satir.rstrip().encode()
    s.sendall(struct.pack('>I', len(p)) + p)
"
```

İki ayrı Python istemcisi açıp birinden "merhaba" yazınca diğerinin de ekranında görmelisin.

Dikkat: Mesaj Birleşmesi/Bölünmesi

TCP bayt akışıdır; `readyRead` bir kerede tam bir mesajın geldiği garantisi vermez. `cerceveCoz` fonksiyonu `while` döngüsüyle birden fazla tam çerçeveyi tek seferde çıkarabilmeli (10 mesaj birden gelmiş olabilir).

4 Bonus Çalışma: JSON Komut Çözücü (30 dk)

Bonus

Bu çalışma zorunlu değildir; tamamlayanlar Çalışma 3'ün payload'larını JSON olarak yorumlayan bir dispatcher inşa eder.

Ön Bilgi

Qt'nin `QJsonDocument` sınıfı bir `QByteArray`'i alıp `QJsonObject`'e parse eder. Hatalı JSON'da `QJsonParseError` ile geri bildirim verir. Bu çalışmada `{"komut":"echo","veri":"..."}` formatındaki istekleri tanıyıp komuta göre cevap üreteceksin.

Görev

`bonus_json/main.cpp` dosyasındaki `komutIsle` fonksiyonunu doldur:

1. `QJsonDocument::fromJson` ile parse et, hata durumunda `{"hata":"gecersiz JSON"}` dönder.
2. `obj.value("komut").toString()` ile komutu al.
3. if/else if zinciri (veya `QHash`) ile dispatch:
   - `"echo"` → `cevap["veri"] = obj.value("veri").toString()`
   - `"ping"` → `cevap["veri"] = "pong"`
   - `"tarih"` → `QDateTime::currentDateTime().toString(Qt::ISODate)`
   - bilinmeyen → `cevap["hata"] = "bilinmeyen komut"`
4. `QJsonDocument(cevap).toJson(QJsonDocument::Compact)` ile `QByteArray` dönder.

Beklenen Çıktı

```
>>> {"komut":"echo","veri":"merhaba OBYS"}
<<< {"veri":"merhaba OBYS"}
----
>>> {"komut":"ping"}
<<< {"veri":"pong"}
----
>>> {"komut":"tarih"}
<<< {"veri":"2026-05-04T..."}
----
>>> {"komut":"silahlanmazu"}
<<< {"hata":"bilinmeyen komut"}
```

Değerlendirme

Çalışma	Puan
Çalışma 1 – POSIX Echo Sunucu/İstemci	30
Çalışma 2 – Qt TCP İstemci	30
Çalışma 3 – Qt Chat + Length-Prefix	30
Bonus – JSON Dispatch	10
Toplam	**100**

Hata Ayıklama

Bağlantı sorunlarında şu komutlar yardımcıdır:

```
# Hangi port hangi süreç tarafından dinleniyor:
lsof -i :9000
ss -tlnp | grep 9000

# TCP trafiğini izle:
sudo tcpdump -i lo port 9000

# Yazılım güvenlik duvarı için:
sudo ufw status
```

Hata "Address already in use" alıyorsan: önceki sunucu süreci hala çalışıyor olabilir (`killall echo_sunucu`) veya `SO_REUSEADDR` koymadın.